

Day 8: Retrieval-Augmented Generation, RAG

1. 5-line beginner summary

RAG means **Retrieval-Augmented Generation**. It helps an LLM answer using **your company documents**, not only its own trained knowledge. RAG first **searches relevant information**, then sends that information to the LLM. This reduces hallucination because the answer is grounded in real sources. RAG is widely used for document Q&A, policy search, support bots, knowledge assistants, and enterprise copilots.

2. What RAG is

A normal LLM generates answers from what it learned during training.

RAG adds one extra step before generation:

| **Retrieve relevant knowledge first, then generate the answer.**

So instead of asking the LLM:

What is our company's leave policy?

We do this:

1. Search leave policy documents.
 2. Find the most relevant paragraphs.
 3. Send those paragraphs to the LLM.
 4. Ask the LLM to answer only using those paragraphs.

That full process is called **RAG**.

Simple meaning:

RAG = Search + LLM Answer

3. Why normal LLMs are not enough for enterprise knowledge

Normal LLMs are powerful, but they have major limitations in companies.

Problem 1: They do not know private company data

An LLM may know public information, but it does not automatically know:

- Internal HR policies
 - Project documents
 - Architecture diagrams
 - Client contracts
 - Incident reports
 - Support tickets
 - Internal emails
 - SOP documents
 - Knowledge base articles

Example:

User: What is the reimbursement limit for my department?

Normal LLM: It may guess.

RAG system: It searches the company's reimbursement policy document and answers from that.

Problem 2: LLM knowledge may be outdated

LLMs are trained up to a certain time. They may not know the latest:

- New policy changes
- New product releases
- Latest pricing
- Updated process documents
- Latest architecture decisions

This is called the **knowledge cutoff problem**.

RAG solves this by connecting the LLM to updated documents.

Problem 3: LLMs can hallucinate

Hallucination means the model gives an answer that sounds confident but is wrong.

Example:

User: What is the notice period in our company?

LLM: The notice period is 30 days.

Reality: The company policy says 60 days.

The answer looks correct, but it is not grounded in the real document.

RAG reduces this by forcing the LLM to answer using retrieved source content.

4. Hallucination problem

Hallucination happens when the LLM fills missing information using probability.

It does not “know” truth like a database. It predicts likely text.

For example:

Question:

What is the escalation process for production incidents in Project Alpha?

LLM without RAG:

Project Alpha incidents should be escalated to the DevOps team within 24 hours.

Problem:
This may be completely invented.

With RAG:

The retriever finds the actual incident escalation SOP.

LLM answer:

As per the Project Alpha incident SOP, P1 incidents must be escalated to the SRE lead within 15 minutes.

Better because the answer is based on actual content.

5. Knowledge cutoff problem

A normal LLM may not know information added after its training.

Example:

Your company updated the leave policy in June 2026.

Normal LLM:

May answer based on general leave policy knowledge.

RAG:

Can search the latest June 2026 policy document.

So RAG is useful when answers depend on changing or private knowledge.

6. How RAG solves document Q&A

Document Q&A means users ask questions from documents.

Examples:

- What does this contract say about termination?
- What are the steps to deploy this application?
- What is the refund policy?
- What are the benefits for employees?
- Which table stores customer metadata?

RAG solves this using this flow:

Documents → Chunks → Embeddings → Vector DB → Retriever → Prompt → LLM → Answer with sources

The LLM does not need to memorize all documents. It only receives the most relevant document pieces during question answering.

7. Easy real-world example

Imagine a company has 500 HR policy PDFs.

An employee asks:

Can I carry forward unused leaves to next year?

Without RAG:

LLM may guess based on common HR practices.

With RAG:

Step 1: Search HR policy documents.
Step 2: Find chunks mentioning "carry forward leave".
Step 3: Send those chunks to the LLM.
Step 4: LLM answers:
"Yes, employees can carry forward up to 10 unused leaves,
as mentioned in the Annual Leave Policy, section 4.2."

This is more reliable because the answer is grounded in the company document.

8. Main components of RAG

8.1 Document ingestion

Document ingestion means collecting and loading documents into the RAG system.

Documents can come from:

- PDFs
- Word files
- Web pages
- SharePoint
- Confluence
- Jira
- GitHub README files
- Database records
- CSV files
- Internal knowledge base

The goal is to convert documents into machine-readable text.

Example:

Input:
HR_policy.pdf

After ingestion:
Plain text extracted from the PDF

8.2 Chunking

LLMs cannot read thousands of pages at once. So documents are split into smaller pieces called **chunks**.

Example document:

Company Leave Policy
1. Annual leave
2. Sick leave
3. Carry forward rules
4. Leave encashment

Possible chunks:

Chunk 1: Annual leave details
Chunk 2: Sick leave details
Chunk 3: Carry forward rules
Chunk 4: Leave encashment rules

Good chunking is very important.

If chunks are too small:

The chunk may lose meaning.

If chunks are too large:

The retriever may bring unnecessary information.

Common chunk sizes:

500 to 1000 tokens

With overlap:

50 to 150 tokens

Overlap means some text is repeated between chunks so context is not lost.

8.3 Embeddings

Embeddings convert text into numbers.

The purpose is to capture the meaning of text.

Example:

Text:
"How many leaves can I carry forward?"

Embedding:
[0.12, -0.45, 0.88, 0.31, ...]

Similar meanings get similar vectors.

Example:

"carry forward unused leaves"
"transfer remaining leave balance to next year"

These two sentences use different words, but their meaning is similar. Embeddings help the system understand that.

8.4 Vector database

A vector database stores embeddings and helps search similar meanings.

Examples of vector databases:

- FAISS
- Chroma
- Pinecone
- Milvus
- Weaviate
- Qdrant
- Elasticsearch with vector search
- Databricks Vector Search

A vector database stores:

- Chunk text
- Chunk embedding
- Document name
- Page number
- Section name
- Created date
- Access permissions
- Source URL

Example record:

```
chunk_id: HR_001_04
text: Employees may carry forward up to 10 unused leaves.
embedding: [0.12, 0.88, -0.24, ...]
metadata:
  document_name: Annual_Leave_Policy.pdf
  page: 4
  section: Carry Forward Rules
```

8.5 Retriever

Retriever searches the vector database and finds relevant chunks for the user question.

Example:

```
Question:
Can I carry forward unused leaves?

Retriever finds:
1. Chunk about leave carry forward
2. Chunk about annual leave limits
3. Chunk about policy exceptions
```

Retriever does not generate the final answer. It only finds relevant information.

Common retrieval methods:

- Semantic search
- Keyword search
- Hybrid search
- Metadata filtering
- Reranking

For basic RAG, semantic search is enough to understand.

8.6 Prompt construction

After relevant chunks are retrieved, we create a prompt for the LLM.

Example prompt:

```
You are an enterprise HR assistant.  
Answer the user question using only the provided context.  
If the answer is not present, say "I do not have enough information."  
  
Context:  
[Chunk 1]  
Employees may carry forward up to 10 unused leaves.  
  
[Chunk 2]  
Carry forward is allowed only for annual leave, not sick leave.  
  
Question:  
Can I carry forward unused leaves?  
  
Answer:
```

Prompt construction is important because it tells the LLM:

- What role to play
- What context to use
- What not to do
- How to format the answer
- Whether to include citations

8.7 LLM response generation

Now the LLM uses the retrieved chunks and generates the final answer.

Example answer:

```
Yes. Employees can carry forward up to 10 unused annual leaves to the next year.  
However, sick leave cannot be carried forward.  
Source: Annual Leave Policy, page 4.
```

The LLM is still generating text, but now it is guided by real source content.

8.8 Citations and source grounding

Citations show where the answer came from.

Example:

Answer:
Employees can carry forward up to 10 unused annual leaves.

Source:
Annual_Leave_Policy.pdf, page 4, section "Carry Forward Rules"

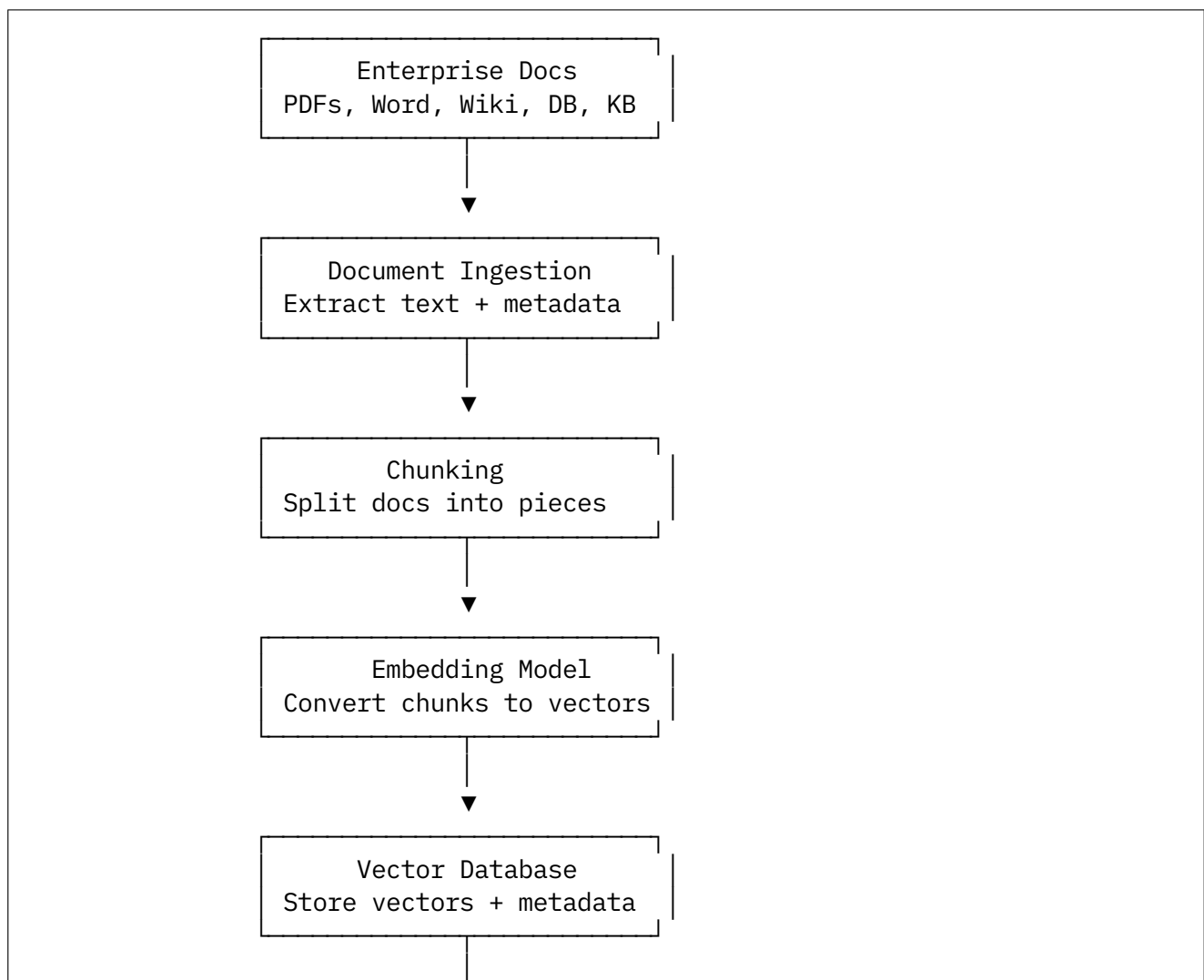
Citations are important in enterprise systems because users need trust.

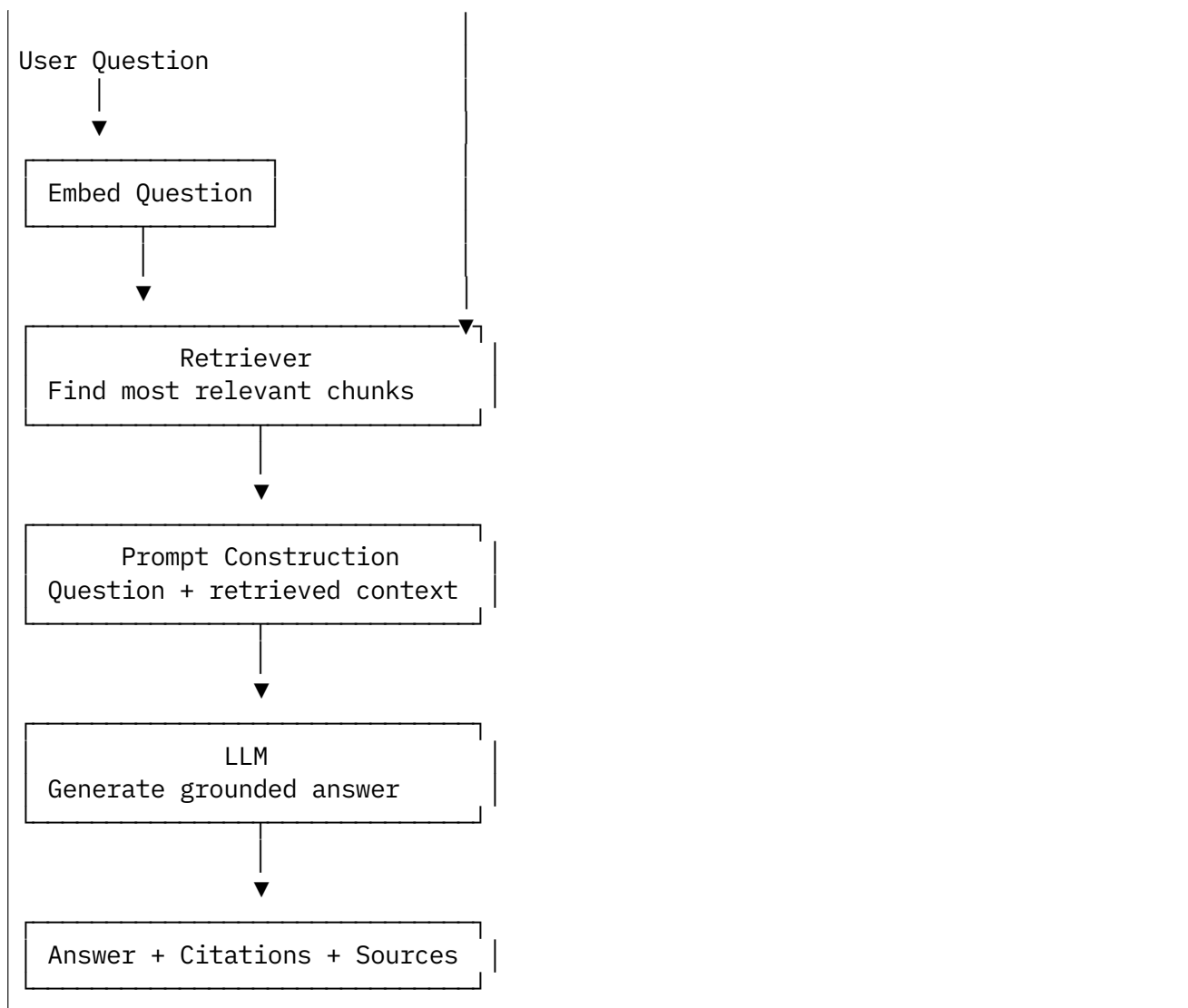
They help answer:

- Where did this answer come from?
- Can I verify it?
- Is this based on real policy?
- Which document was used?

A good RAG system should not only answer. It should also show the source.

9. ASCII diagram showing RAG architecture





10. Basic RAG pipeline in simple stages

There are two major parts:

1. Offline indexing pipeline
2. Online question-answering pipeline

10.1 Offline indexing pipeline

This happens before users ask questions.

Documents are processed and stored in vector database.

Flow:

Load documents
→ Extract text
→ Split into chunks
→ Create embeddings
→ Store chunks and embeddings in vector DB

10.2 Online question-answering pipeline

This happens when user asks a question.

```
User asks question
→ Convert question into embedding
→ Search vector DB
→ Retrieve relevant chunks
→ Build prompt
→ Send prompt to LLM
→ Return answer with sources
```

11. Pseudocode for basic RAG pipeline

11.1 Indexing documents

```
FUNCTION index_documents(document_folder):

    documents = load_all_documents(document_folder)

    FOR each document IN documents:

        text = extract_text(document)

        metadata = {
            "document_name": document.name,
            "source_path": document.path,
            "created_date": document.created_date
        }

        chunks = split_text_into_chunks(
            text,
            chunk_size = 800 tokens,
            overlap = 100 tokens
        )

        FOR each chunk IN chunks:

            embedding = create_embedding(chunk.text)

            vector_database.store(
                id = unique_chunk_id,
                vector = embedding,
                text = chunk.text,
                metadata = metadata + chunk.page_number
            )

    RETURN "Documents indexed successfully"
```

11.2 Answering user questions

```
FUNCTION answer_question(user_question):

    question_embedding = create_embedding(user_question)

    retrieved_chunks = vector_database.search(
        query_vector = question_embedding,
        top_k = 5
    )

    context = combine_retrieved_chunks(retrieved_chunks)

    prompt = build_prompt(
        instruction = "Answer only using the provided context. If answer is
missing, say you do not know.",
        context = context,
        question = user_question
    )

    llm_answer = call_llm(prompt)

    sources = extract_sources(retrieved_chunks)

    final_response = {
        "answer": llm_answer,
        "sources": sources
    }

    RETURN final_response
```

12. Simple Python-like pseudocode

```
# Step 1: Load documents
documents = load_documents("company_policies/")

# Step 2: Split documents into chunks
all_chunks = []

for document in documents:
    text = extract_text(document)
    chunks = split_text(text, chunk_size=800, overlap=100)

    for chunk in chunks:
        all_chunks.append({
            "text": chunk,
            "document_name": document.name
        })

# Step 3: Convert chunks into embeddings
for chunk in all_chunks:
    chunk["embedding"] = embedding_model.embed(chunk["text"])
```

```

# Step 4: Store embeddings in vector database
vector_db.insert(all_chunks)

# Step 5: User asks question
question = "Can employees carry forward unused leaves?"

# Step 6: Convert question into embedding
question_embedding = embedding_model.embed(question)

# Step 7: Retrieve relevant chunks
relevant_chunks = vector_db.search(question_embedding, top_k=5)

# Step 8: Build prompt
context = "\n".join([chunk["text"] for chunk in relevant_chunks])

prompt = f"""
Answer the question using only the context below.

Context:
{context}

Question:
{question}
"""

# Step 9: Generate answer
answer = llm.generate(prompt)

# Step 10: Return answer with sources
print(answer)
print("Sources:", [chunk["document_name"] for chunk in relevant_chunks])

```

13. Basic RAG evaluation

RAG evaluation checks whether the system is answering correctly and using the right sources.

There are two parts:

1. Retrieval evaluation
2. Generation evaluation

13.1 Retrieval evaluation

This checks whether the retriever found the right chunks.

Example:

```

Question:
Can employees carry forward unused leaves?

```

Expected source:
Annual Leave Policy, page 4

Retrieved source:
Annual Leave Policy, page 4

This is good retrieval.

Bad retrieval:

Retrieved source:
Sick Leave Policy, page 2

Common retrieval metrics:

Precision:
Out of retrieved chunks, how many were useful?

Recall:
Out of all useful chunks, how many did we retrieve?

Top-k accuracy:
Was the correct chunk present in the top 3 or top 5 results?

13.2 Generation evaluation

This checks whether the LLM answered correctly using the retrieved context.

Good answer:

The answer is correct, complete, and supported by source text.

Bad answer:

The answer is fluent but not supported by source text.

Generation quality can be checked using:

- Correctness
- Faithfulness
- Completeness
- Citation accuracy
- Answer relevance

13.3 Simple evaluation table

Evaluation area	Question to ask
Retrieval quality	Did we retrieve the right chunks?
Answer correctness	Is the answer factually correct?
Faithfulness	Is the answer supported by the retrieved context?

Evaluation area	Question to ask
Citation quality	Are the cited sources correct?
Completeness	Did the answer cover the full question?
Safety	Did the model avoid unsupported claims?

14. SQL/NoSQL/database connection with RAG

Since you already covered databases on Day 7, connect it like this:

Storage type	Role in RAG
Object storage	Stores original PDFs, Word files, images
SQL database	Stores metadata, users, access permissions, logs
NoSQL database	Stores flexible document metadata or chat history
Vector database	Stores embeddings for semantic search
Cache	Stores repeated question results for faster response

Example:

PDF file	→ Object storage
Document metadata	→ SQL database
Chunk text	→ NoSQL or vector DB
Chunk embedding	→ Vector database
User query logs	→ SQL/NoSQL database
LLM response history	→ SQL/NoSQL database

15. RAG in enterprise AI systems

Enterprise RAG needs more than a simple demo.

Important enterprise concerns:

- Access control - Data security - Source citation - Document freshness - Logging and monitoring - Evaluation - Cost control - Latency control - Model governance - User feedback
--

Example:

An employee from Finance should not retrieve HR confidential documents.
A junior developer should not retrieve client contract pricing unless permitted.

So metadata filtering is important.

Example metadata:

```
department: HR
access_level: employee
region: India
document_type: policy
last_updated: 2026-06-01
```

The retriever can use filters:

```
Only retrieve documents where:
department = user's department
access_level <= user's permission
region = user's region
```

16. Simple comparison: normal LLM vs RAG

Topic	Normal LLM	RAG system
Uses private documents	No	Yes
Handles latest knowledge	Limited	Yes, if documents are updated
Reduces hallucination	Not always	Better grounding
Gives citations	Usually no	Yes
Useful for enterprise Q&A	Limited	Strong
Needs vector DB	No	Usually yes
Needs document pipeline	No	Yes

17. Common mistakes

Mistake 1: Thinking RAG removes hallucination completely

RAG reduces hallucination, but it does not fully remove it.

Why?

- Retriever may fetch wrong chunks
- Prompt may be weak
- LLM may ignore context
- Documents may be outdated
- Chunks may be incomplete

Better approach:

Use strong prompts, good retrieval, citations, evaluation, and fallback responses.

Mistake 2: Poor chunking

Bad chunking can destroy RAG quality.

Example bad chunk:

"Employees may carry forward..."

This is incomplete.

Better chunk:

"Employees may carry forward up to 10 unused annual leaves to the next calendar year.
Sick leave cannot be carried forward."

A chunk should contain enough meaning by itself.

Mistake 3: Storing embeddings without metadata

If you store only vectors, you lose source information.

Bad:

embedding only

Good:

embedding + chunk text + document name + page number + section + access level

Metadata helps with:

- Citations
- Filtering
- Security
- Debugging
- Evaluation

Mistake 4: Sending too many chunks to the LLM

More context is not always better.

Too many chunks can cause:

- Higher cost
- Slower response
- Confused answers
- Irrelevant information in prompt

Usually retrieve top 3 to top 10 chunks depending on use case.

Mistake 5: Not checking source quality

If the documents are poor, the answer will be poor.

RAG follows this rule:

Bad documents → bad retrieval → bad answer

So document cleaning is important.

Mistake 6: No evaluation

Many beginners build RAG demos but do not test them properly.

You should test:

- Did the retriever find the right chunk?
 - Did the answer match the source?
 - Did the citation point to the correct document?
 - Did the system say "I do not know" when answer was missing?
-

Mistake 7: Ignoring access control

Enterprise RAG must respect permissions.

Bad:

All users can search all documents.

Good:

Retriever filters documents based on user role, department, and access level.

This is very important in IBM-style enterprise AI projects.

18. Final revision sheet

RAG = Retrieval-Augmented Generation

Purpose:

Use company documents to answer user questions.

Main steps:

1. Ingest documents
2. Split into chunks
3. Create embeddings
4. Store in vector database
5. Embed user question
6. Retrieve relevant chunks
7. Build prompt
8. Generate answer using LLM
9. Return citations

10. Evaluate quality

Main benefits:

- Reduces hallucination
- Uses private enterprise data
- Handles updated knowledge
- Provides source grounding
- Improves trust

Main risks:

- Wrong retrieval
- Poor chunking
- Missing citations
- Outdated documents
- No access control
- No evaluation

19. One-line interview answer

RAG is an architecture where we first retrieve relevant information from enterprise documents using embeddings and vector search, then pass that retrieved context to an LLM so it can generate a grounded, source-backed answer instead of relying only on its trained knowledge.